

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра “Вычислительная техника”

Отчёт

по лабораторной работе №5
по курсу “Программирование на языке Java”
на тему “Многопоточность в Java”
Вариант 9

Выполнили студенты гр. 23ВВП1:
Лобачева К.С.
Зонь М.А.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., доцент Карамышева Н.С.

Пенза 2026

Цель работы

Научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Лабораторное задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в семи дополнительных потоках, снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Для распараллеливания вычислений необходимо реализовать интерфейс Runnable.

Ход выполнения работы

В лабораторной работе модифицировано приложение для вычисления определенного интеграла функции $\cos(x^2)$. Для распараллеливания вычислений класс RecIntegral реализует интерфейс Runnable. В рамках данного класса реализован метод run, выполняющий роль отдельного потока. Метод осуществляет вычисление части интеграла методом трапеций на заданном интервале и сохраняет результат в поле res (рис. 1).

```
public void run() {
    double sum = 0.0;
    double x = lowerLimit;

    while (x < upperLimit) {
        double nextX = Math.min(x + step, upperLimit);
        double y1 = Math.cos(x * x);
        double y2 = Math.cos(nextX * nextX);
        sum += (y1 + y2) * (nextX - x) / 2.0;
        x = nextX;
    }
    this.res = sum;
}
```

Рисунок 1 — Метод run класса RecIntegral

Для экземпляров вычислительного класса и потоков созданы массивы фиксированного размера. Исходный интервал интегрирования равномерно разбивается на 9 подинтервалов. Создание и запуск вычислительных потоков организованы в цикле в методе Rasschet основного класса l2frame (рис. 2).

```

new Thread(() -> {
    try {

        double interval = (upperLimit - lowerLimit) / 9;
        Thread[] threads = new Thread[9];
        RecIntegral[] calculators = new RecIntegral[9];

        for (int i = 0; i < 9; i++) {
            double threadLower = lowerLimit + i * interval;
            double threadUpper = threadLower + interval;
            calculators[i] = new RecIntegral(threadLower, threadUpper, step);
            threads[i] = new Thread(calculators[i]);
            threads[i].start();
        }

        long starttime = System.nanoTime();
        double total = 0.0;
        for (int i = 0; i < 9; i++) {
            threads[i].join();
            total += calculators[i].getRes();
        }
    }
}

```

Рисунок 2 — Создание и организация вычислительных потоков

В качестве инструмента синхронизации использовался метод барьерной синхронизации `join()`.

Результат работы программы представлен на рисунке 3.

Function: $\cos(x^2)$

Данные для интегрирования:

Верхний предел:

Нижний предел:

Шаг:

Нижний предел	Верхний предел	Шаг	Результат
1.0	100000.0	1.0	-272,513294
0.001	1000.0	1.0	12,283876

Рисунок 3 — Результат работы программы

Вывод

В ходе выполнения данной лабораторной работы были получены навыки организации многопоточных программ на языке Java. Изучен механизм синхронизации вычислительных потоков `join`.